

# Manual do Desenvolvedor

## IgnisGeo — Sistema Web de Priorização Geoespacial de Áreas de Risco de Queimadas

V1.2026

---

Guia completo de configuração e execução do ambiente de desenvolvimento local em  
Ubuntu / Linux Mint.

### **Cobertura:**

- ✓ Git clone e configuração inicial
- ✓ Pré-requisitos: Docker, Python, Node.js
- ✓ Variáveis de ambiente (.env)
- ✓ Docker Compose: 5 containers
- ✓ Migrações e banco de dados PostGIS
- ✓ Exportação do CSV do INPE BDQueimadas
- ✓ Importação incremental dos dados
- ✓ Execução do TOPSIS Fuzzy
- ✓ Testes automatizados (46 testes)
- ✓ Validação dos endpoints com curl
- ✓ Documentação pdoc
- ✓ Comandos de manutenção

---

Rogério Cerqueira  
Engenharia de Computação — UEFS · TCC 2026.1

Repositório: <https://github.com/rogeriocerqueira/ignisGeo>

## Contents

---

|           |                                                         |           |
|-----------|---------------------------------------------------------|-----------|
| <b>1</b>  | <b>Pré-requisitos</b>                                   | <b>3</b>  |
| 1.1       | Verificar versões instaladas . . . . .                  | 3         |
| 1.2       | Instalar Docker e Docker Compose V2 . . . . .           | 3         |
| 1.3       | Instalar Python 3.12 e pip . . . . .                    | 3         |
| 1.4       | Instalar Node.js 20 via nvm . . . . .                   | 3         |
| <b>2</b>  | <b>Git Clone e Configuração Inicial</b>                 | <b>4</b>  |
| <b>3</b>  | <b>Ambiente Virtual Python (backend local)</b>          | <b>5</b>  |
| <b>4</b>  | <b>Docker Compose — Subindo os Containers</b>           | <b>6</b>  |
| 4.1       | Comandos Docker essenciais . . . . .                    | 6         |
| <b>5</b>  | <b>Banco de Dados — Migrações e PostGIS</b>             | <b>7</b>  |
| 5.1       | Verificar extensão PostGIS . . . . .                    | 7         |
| 5.2       | Verificar tabelas criadas . . . . .                     | 7         |
| <b>6</b>  | <b>Exportação do CSV do INPE BDQueimadas</b>            | <b>8</b>  |
| 6.1       | Estrutura do CSV do INPE . . . . .                      | 8         |
| <b>7</b>  | <b>Importação dos Dados</b>                             | <b>8</b>  |
| <b>8</b>  | <b>Execução do TOPSIS Fuzzy</b>                         | <b>11</b> |
| 8.1       | TOPSIS com filtros . . . . .                            | 12        |
| <b>9</b>  | <b>Validação dos Endpoints com curl</b>                 | <b>12</b> |
| 9.1       | GET /api/ranking/ — Ranking paginado . . . . .          | 12        |
| 9.2       | GET /api/focos/geojson/ — GeoJSON dos focos . . . . .   | 12        |
| 9.3       | GET /api/serie-temporal/ — Série mensal . . . . .       | 12        |
| 9.4       | GET /api/graficos-dados/ — Dados dos gráficos . . . . . | 13        |
| 9.5       | GET /api/correlacao/ — Matriz de correlação . . . . .   | 13        |
| <b>10</b> | <b>Testes Automatizados</b>                             | <b>13</b> |
| 10.1      | Testes com cobertura de código . . . . .                | 14        |
| <b>11</b> | <b>Frontend — Vue 3</b>                                 | <b>14</b> |
| 11.1      | Acessar a interface . . . . .                           | 14        |
| 11.2      | Desenvolvimento frontend local (sem Docker) . . . . .   | 14        |
| <b>12</b> | <b>Documentação pdoc</b>                                | <b>14</b> |
| <b>13</b> | <b>Comandos de Manutenção</b>                           | <b>15</b> |
| 13.1      | Banco de dados . . . . .                                | 15        |
| 13.2      | Git — Fluxo de trabalho . . . . .                       | 16        |
| 13.3      | Limpeza do ambiente Docker . . . . .                    | 16        |
| <b>14</b> | <b>Resolução de Problemas Comuns</b>                    | <b>16</b> |

---

|                                                 |           |
|-------------------------------------------------|-----------|
| 14.1 Container <b>db</b> não inicia . . . . .   | 16        |
| 14.2 Celery não conecta ao Redis . . . . .      | 16        |
| 14.3 PostGIS não disponível . . . . .           | 16        |
| 14.4 Frontend não conecta ao backend . . . . .  | 17        |
| 14.5 Importação falha silenciosamente . . . . . | 17        |
| <b>15 Checklist Final — Sistema Rodando</b>     | <b>17</b> |
| <b>16 Recuperação de Senha do Admin</b>         | <b>18</b> |
| <b>17 Resumo dos Endereços Locais</b>           | <b>18</b> |

## 1. Pré-requisitos

Antes de clonar o projeto, o ambiente deve ter as ferramentas abaixo instaladas. Todos os comandos foram validados em Ubuntu 22.04 LTS e Linux Mint 21.

### 1.1. Verificar versões instaladas

```
git      --version    # >= 2.34
docker  --version    # >= 24.0
docker  compose version # >= 2.20 (sem hífen - V2)
python3 --version    # >= 3.10
node    --version    # >= 18.0
npm      --version    # >= 9.0
curl    --version    # >= 7.80
```

### 1.2. Instalar Docker e Docker Compose V2

```
sudo apt remove docker docker-engine docker.io containerd runc
sudo apt update
sudo apt install -y ca-certificates curl gnupg lsb-release
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli \
  containerd.io docker-compose-plugin
sudo usermod -aG docker $USER
newgrp docker
docker --version
docker compose version
```

**Esperado:**

Docker version 24.x.x

Docker Compose version v2.x.x

### 1.3. Instalar Python 3.12 e pip

```
sudo apt update
sudo apt install -y python3.12 python3.12-venv python3-pip
python3 --version    # Python 3.12.x
```

### 1.4. Instalar Node.js 20 via nvm

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/  
install.sh \  
| bash  
source ~/.bashrc  
nvm install 20  
nvm use 20  
node --version    # v20.x.x
```

## 2. Git Clone e Configuração Inicial

### Passo 1 — Clone do repositório

```
git clone https://github.com/rogeriocerqueira/ignisGeo.git  
cd ignisGeo  
ls -la
```

#### Estrutura esperada:

```
drwxr-xr-x  backend/          # Django + DRF + Celery  
drwxr-xr-x  frontend/        # Vue 3 + Vite  
drwxr-xr-x  data/             # CSVs do INPE (volume Docker)  
drwxr-xr-x  docs/             # Diagramas e documentacao  
-rw-r--r--  docker-compose.yml  
-rw-r--r--  .gitignore  
-rw-r--r--  README.md
```

### Passo 2 — Criar arquivo de variáveis de ambiente

O arquivo `.env` nunca é versionado (está no `.gitignore`). Crie na raiz do projeto:

```
cat > .env << 'EOF'  
POSTGRES_DB=queimadas  
POSTGRES_USER=queimadas_user  
POSTGRES_PASSWORD=queimadas_pass  
DB_HOST=db  
DB_PORT=5432  
DJANGO_SECRET_KEY=troque-por-uma-chave-forte  
DJANGO_DEBUG=True  
REDIS_URL=redis://redis:6379/0  
VITE_API_BASE_URL=http://localhost:8000/api  
EOF
```

#### Conteúdo do `.env`:

```
# Banco de dados  
POSTGRES_DB=queimadas  
POSTGRES_USER=queimadas_user  
POSTGRES_PASSWORD=queimadas_pass  
DB_HOST=db
```

```
DB_PORT=5432

# Django
DJANGO_SECRET_KEY=troque-por-uma-chave-forte
DJANGO_DEBUG=True

# Redis / Celery
REDIS_URL=redis://redis:6379/0

# Frontend
VITE_API_BASE_URL=http://localhost:8000/api
```

**Atenção:** nunca commite o arquivo `.env` no Git. Ele contém credenciais e está protegido pelo `.gitignore`.

### 3. Ambiente Virtual Python (backend local)

#### Passo 3 — Criar e ativar o ambiente virtual

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
which python
# Esperado: .../ignisGeo/backend/.venv/bin/python
pip install -r requirements.txt
```

**GDAL no Ubuntu:** se `pip install gdal` falhar, instale a biblioteca do sistema antes:

```
sudo apt install -y gdal-bin libgdal-dev
export CPL_INCLUDE_PATH=/usr/include/gdal
export C_INCLUDE_PATH=/usr/include/gdal
pip install gdal==$(gdal-config --version)
```

**Lembre-se:** em cada novo terminal, ative o ambiente virtual e exporte a variável do banco antes de rodar comandos Django localmente:

```
source backend/.venv/bin/activate
export DATABASE_URL=postgres://queimadas_user:
    queimadas_pass@localhost:5433/queimadas
```

## 4. Docker Compose — Subindo os Containers

### Passo 4 — Build e inicialização dos containers

```
# A partir da raiz do projeto (ignisGeo/)
cd ..
docker compose build
docker compose up -d
docker compose logs -f
```

### Passo 5 — Verificar containers em execução

```
docker compose ps
```

Saída esperada:

```
NAME SERVICE STATUS PORTS
ignisgeo_backend-1 backend running 0.0.0.0:8000->8000/tcp
ignisgeo_frontend-1 frontend running 0.0.0.0:5173->5173/tcp
ignisgeo_db-1 db healthy 0.0.0.0:5433->5432/tcp
ignisgeo_redis-1 redis running 0.0.0.0:6379->6379/tcp
ignisgeo_celery-1 celery running
```

Todos os 5 containers devem estar com STATUS = running.

Se algum aparecer exited, verifique os logs:

```
docker compose logs <nome-do-servico>
```

### 4.1. Comandos Docker essenciais

```
# Para todos os containers (mant m os dados)
docker compose down

# Para e remove volumes (APAGA o banco)
docker compose down -v

# Reinicia apenas o backend
docker compose restart backend

# Reinicia backend e celery (ap s alterar .env)
docker compose restart backend celery

# Acessa o shell dentro do container backend
docker compose exec backend bash

# Acessa o psql dentro do container do banco
docker compose exec db psql -U queimadas_user -d queimadas

# Logs de um container espec fico
```

```
docker compose logs backend -f
docker compose logs celery -f
docker compose logs db -f

# Rebuild ap s mudan as no c digo
docker compose up -d --build backend
```

## 5. Banco de Dados — Migrações e PostGIS

### Passo 6 — Aplicar migrações Django

```
docker compose exec backend python manage.py migrate
```

Saída esperada:

```
Applying queimadas.0001_initial... OK
Applying queimadas.0002_alter_arearisco_geometria... OK
Applying queimadas.0003_rename_ndvi_medio... OK
```

### 5.1. Verificar extensão PostGIS

```
docker compose exec db psql -U queimadas_user -d queimadas \
-c "SELECT postgis_version();"
```

```
postgis_version
-----
3.4 USE_GEOS=1 USE_PROJ=1 USE_STATS=1
```

### 5.2. Verificar tabelas criadas

```
docker compose exec db psql -U queimadas_user -d queimadas \
-c "\dt"
```

| Schema | Name                   | Type  | Owner          |
|--------|------------------------|-------|----------------|
| public | queimadas_arearisco    | table | queimadas_user |
| public | queimadas_focoqueimada | table | queimadas_user |
| public | spatial_ref_sys        | table | queimadas_user |

### Passo 7 — Criar superusuário Django (opcional)

```
docker compose exec backend python manage.py createsuperuser
# Username: admin
# Email: admin@ignisGeo.local
# Password: (escolha uma senha)
# Admin em: http://localhost:8000/admin/
```



## 6. Exportação do CSV do INPE BDQueimadas

### Passo 8 — Exportar dados do BDQueimadas

Acesse o portal do INPE em: <https://queimadas.dgi.inpe.br/queimadas/bdqueimadas>

**Parâmetros de exportação utilizados no IgnisGeo:**

| Parâmetro      | Valor                    |
|----------------|--------------------------|
| País           | Brasil                   |
| Estados        | Todos (26 estados + DF)  |
| Biomass        | Todos                    |
| Período início | 01/01/2024               |
| Período fim    | 30/04/2026               |
| Formato        | CSV                      |
| Satélite ref.  | Aqua Tarde (padrão INPE) |

Coloque os arquivos baixados na pasta `data/` na raiz do projeto. O nome do arquivo não importa — o script de importação processa qualquer `.csv` presente na pasta. Sugestão de organização:

```
data/
focos_2024.csv    # jan/2024 - dez/2024
focos_2025.csv    # jan/2025 - dez/2025
focos_2026.csv    # jan/2026 - abr/2026
```

### 6.1. Estrutura do CSV do INPE

```
head -2 data/focos_2024.csv
```

```
id,latitude,longitude,data_hora_gmt,satelite,municipio,estado,
pais,municipio_id,bioma,frp,risco_fogo,numero_dias_sem_chuva,
precipitacao
1,-10.59,-45.12,2024-01-01 13:24:00,Aqua_Tarde,CORURUPE,AL,...
```

## 7. Importação dos Dados

O IgnisGeo utiliza um script de importação incremental que lê todos os arquivos `.csv` da pasta `data/` e insere apenas os registros ainda não existentes no banco, ignorando duplicatas via `ignore_conflicts=True`.

A pasta local `./data/` está mapeada como volume Docker:

```
# Pasta local          Container
```

```
./data/          -->  /app/data/

# Verificar arquivos dispon veis dentro do container
docker compose exec backend ls -lh /app/data/
```

O script importa **todos** os **.csv** da pasta automaticamente. O nome do arquivo não importa — apenas a extensão é verificada. Para adicionar dados de um novo período, basta colocar o CSV na pasta **data/** e rodar o script novamente.

### Passo 9 — Importação incremental

```
docker compose exec backend python manage.py shell -c "
import csv
from queimadas.models import FocoQueimada
from queimadas.tasks import parse_linha
import os

pasta = '/app/data/'
total = 0

for arquivo in sorted(os.listdir(pasta)):
    if not arquivo.endswith('.csv'):
        continue
    print(f'Importando {arquivo}...')
    caminho = pasta + arquivo
    lote = []
    try:
        with open(caminho, encoding='utf-8-sig') as f:
            for linha in csv.DictReader(f):
                dados = parse_linha(linha)
                if dados:
                    lote.append(FocoQueimada(**dados))
                if len(lote) >= 5000:
                    FocoQueimada.objects.bulk_create(
                        lote, ignore_conflicts=True)
                    total += len(lote)
                    lote = []
                    print(f'  {total:,} importados...',
                        flush=True)
            if lote:
                FocoQueimada.objects.bulk_create(
                    lote, ignore_conflicts=True)
                total += len(lote)
                print(f'OK: {arquivo}')
    except Exception as e:
        print(f'ERRO {arquivo}: {e}')

print(f'Novos: {total:,}')
print(f'Banco: {FocoQueimada.objects.count():,}')
"
```

**Como funciona o `ignore_conflicts=True`:** registros já existentes no banco são silenciosamente ignorados. O script pode ser executado múltiplas vezes sobre o mesmo arquivo sem duplicar dados — ideal para reimportar após `docker compose down -v`.

#### Passo 10 — Monitorar progresso em tempo real

Abra um segundo terminal e rode:

```
watch -n 10 "docker compose exec backend \  
python manage.py shell -c \  
'from queimadas.models import FocoQueimada; \  
print(FocoQueimada.objects.count())'"
```

A contagem atualiza a cada 10 segundos. Quando parar de crescer a importação terminou.

#### Passo 11 — Verificar total importado

```
curl -s http://localhost:8000/api/estatisticas/ \  
| python3 -m json.tool
```

Saída esperada (com dados completos 2024–2026):

```
{  
  "total_focos": 11807488,  
  "areas_criticas": 0,  
  "por_bioma": [  
    {"bioma": "AMAZONIA",      "total": 5530140},  
    {"bioma": "CERRADO",      "total": 4121413},  
    {"bioma": "CAATINGA",     "total": 803330},  
    {"bioma": "MATA_ATLANTICA", "total": 649737},  
    {"bioma": "PANTANAL",     "total": 638415},  
    {"bioma": "PAMPA",        "total": 14237}  
  ]  
}
```

**Quando usar `TRUNCATE`:** apenas para reimportar tudo do zero, por exemplo após corrigir um erro no `parse_linha`:

```
docker compose exec db psql \  
-U queimadas_user -d queimadas \  
-c "TRUNCATE queimadas_focoqueimada  
RESTART IDENTITY CASCADE;"
```

## 8. Execução do TOPSIS Fuzzy

### Passo 12 — Medir desempenho do pipeline

```
docker compose exec backend python manage.py shell -c "  
import time  
from queimadas.topsis_fuzzy import calcular_topsis_fuzzy  
from queimadas.models import FocoQueimada  
from django.db.models import Count, Avg  
  
inicio = time.time()  
dados = list(  
    FocoQueimada.objects.values('municipio', 'estado', 'bioma')  
    .annotate(  
        total_focos=Count('id'),  
        frp_media=Avg('frp'),  
        risco_historico_medio=Avg('risco_historico'),  
        dias_sem_chuva_medio=Avg('dias_sem_chuva'),  
        precipitacao_medio=Avg('precipitacao'),  
    )  
)  
t_sql = time.time() - inicio  
print(f'Agregacao SQL: {t_sql:.3f}s -- {len(dados)}  
municipios')  
  
inicio = time.time()  
ranking = calcular_topsis_fuzzy(dados)  
t_topsis = time.time() - inicio  
print(f'TOPSIS Fuzzy: {t_topsis:.3f}s -- {len(ranking)}  
areas')  
print()  
print('Top 5:')  
for r in ranking[:5]:  
    print(f'    #{r["ranking"]} {r["municipio"]}/{r["  
estado"]}  
f'CC={r["score_topsis"]} {r["nivel_risco"]}')  
"
```

### Resultado medido (conjunto completo 2024–2026):

Agregacao SQL: 1.381s - 6261 municipios  
TOPSIS Fuzzy: 8.305s - 6261 areas

#### Top 5:

#1 ALTAMIRA/PA CC=0.5872 CRITICO  
#2 CORUMBA/MS CC=0.5799 CRITICO  
#3 SAO FELIX DO XINGU/PA CC=0.5772 CRITICO  
#4 APUI/AM CC=0.5749 CRITICO  
#5 MONTE ALEGRE DE MINAS/MG CC=0.5724 CRITICO

**Passo 13 — Executar o pipeline TOPSIS via endpoint**

```
curl -s -X POST http://localhost:8000/api/calcular-topsis/ \
-H "Content-Type: application/json" \
-d '{}' \
| python3 -m json.tool
```

**8.1. TOPSIS com filtros**

```
# Apenas Cerrado
curl -s -X POST http://localhost:8000/api/calcular-topsis/ \
-H "Content-Type: application/json" \
-d '{"bioma": "CERRADO"}' \
| python3 -m json.tool

# Apenas Bahia no período crítico
curl -s -X POST http://localhost:8000/api/calcular-topsis/ \
-H "Content-Type: application/json" \
-d '{
  "estado": "BA",
  "data_inicio": "2024-07-01",
  "data_fim": "2024-12-31"
}' \
| python3 -m json.tool
```

## 9. Validação dos Endpoints com curl

**9.1. GET /api/ranking/ — Ranking paginado**

```
curl -s "http://localhost:8000/api/ranking/?page=1&page_size=5" \
| python3 -m json.tool

# Filtros disponíveis
curl -s "http://localhost:8000/api/ranking/?nivel_risco=CRITICO" \
| python3 -m json.tool
curl -s "http://localhost:8000/api/ranking/?estado=PA" \
| python3 -m json.tool
curl -s "http://localhost:8000/api/ranking/?bioma=AMAZONIA" \
| python3 -m json.tool
```

**9.2. GET /api/focos/geojson/ — GeoJSON dos focos**

```
curl -s "http://localhost:8000/api/focos/geojson/?estado=PA" \
| python3 -m json.tool | head -40
```

**9.3. GET /api/serie-temporal/ — Série mensal**

```
curl -s "http://localhost:8000/api/serie-temporal/" \
| python3 -m json.tool
```

#### 9.4. GET /api/graficos-dados/ — Dados dos gráficos

```
# Todos os datasets para os 8 graficos em 1 chamada
curl -s "http://localhost:8000/api/graficos-dados/" \
| python3 -m json.tool | head -60
```

#### 9.5. GET /api/correlacao/ — Matriz de correlação

```
curl -s "http://localhost:8000/api/correlacao/" \
| python3 -m json.tool
```

Saída esperada (valores reais com N=6.261):

```
{
  "rotulos": [
    "C1 - Focos", "C2 - FRP",
    "C3 - Risco Hist.",
    "C4 - Dias s/ Chuva",
    "C5 - Precipitacao"
  ],
  "pearson": [
    [1.0, 0.1708, 0.0755, 0.0797, -0.0259],
    [0.1708, 1.0, 0.3561, 0.2043, -0.0529],
    [0.0755, 0.3561, 1.0, 0.4748, -0.2492],
    [0.0797, 0.2043, 0.4748, 1.0, -0.1449],
    [-0.0259, -0.0529, -0.2492, -0.1449, 1.0]
  ],
  "n": 6261
}
```

## 10. Testes Automatizados

### Passo 14 — Executar a suite completa de testes

```
# 46 testes (requer containers rodando)
docker compose exec backend \
python manage.py test queimadas -v 2
```

Saída esperada:

```
Found 46 test(s).
...
Ran 46 tests in 0.330s
OK
```

**Passo 15 — Executar apenas testes de sensibilidade OAT**

```
# Nao precisa de banco (SimpleTestCase)
docker compose exec backend \
    python manage.py test queimadas.test_sensibilidade -v 2
```

**Saída esperada:**

```
Found 5 test(s).
Skipping setup of unused database(s): default.
...
RELATORIO DE SENSIBILIDADE -- IGNISGEO (OAT)
Simulacoes: 25
Estab. Top 1: 100.0% (25/25)
Estab. Top 3: 84.0% (21/25)
...
Ran 5 tests in 0.005s
OK
```

**10.1. Testes com cobertura de código**

```
docker compose exec backend bash -c \
    "pip install coverage -q && \
    coverage run manage.py test queimadas && \
    coverage report -m"
```

**11. Frontend — Vue 3****11.1. Acessar a interface**

Interface web: <http://localhost:5173>  
API Django: <http://localhost:8000/api/>  
Admin Django: <http://localhost:8000/admin/>

**11.2. Desenvolvimento frontend local (sem Docker)**

```
cd frontend
npm install
npm run dev
# http://localhost:5173 -- hot-reload ativo
```

**12. Documentação pdoc**

```
docker compose exec backend bash -c \
    "pip install pdoc -q && \
```

```
pdoc queimadas --output-dir /app/docs/"

docker compose cp backend:/app/docs ./backend/docs
xdg-open backend/docs/queimadas/topsis_fuzzy.html
```

## 13. Comandos de Manutenção

---

### 13.1. Banco de dados

```
# Criar novas migracoes apos alterar models.py
docker compose exec backend \
    python manage.py makemigrations queimadas

# Aplicar migracoes
docker compose exec backend \
    python manage.py migrate

# Contar focos no banco
docker compose exec db psql -U queimadas_user -d queimadas \
    -c "SELECT COUNT(*) FROM queimadas_focoqueimada;"

# Distribuicao por semestre
docker compose exec db psql -U queimadas_user -d queimadas \
    -P pager=off -c "
SELECT
    CASE
        WHEN data_hora >= '2024-01-01'
            AND data_hora < '2024-07-01' THEN '1S/2024'
        WHEN data_hora >= '2024-07-01'
            AND data_hora < '2025-01-01' THEN '2S/2024'
        WHEN data_hora >= '2025-01-01'
            AND data_hora < '2025-07-01' THEN '1S/2025'
        WHEN data_hora >= '2025-07-01'
            AND data_hora < '2026-01-01' THEN '2S/2025'
        ELSE '1S/2026'
    END AS semestre,
    COUNT(*) AS focos
FROM queimadas_focoqueimada
GROUP BY semestre
ORDER BY MIN(data_hora);"

# Backup do banco
docker compose exec db pg_dump \
    -U queimadas_user queimadas > backup_$(date +%Y%m%d).sql

# Restaurar backup
docker compose exec -T db psql \
    -U queimadas_user queimadas < backup_20260101.sql
```



### 13.2. Git — Fluxo de trabalho

```
git checkout -b feat/nova-funcionalidade
git add backend/queimadas/views.py
git commit -m "feat(api): adiciona filtro por bioma no ranking"
git push origin feat/nova-funcionalidade
git checkout main
git merge feat/nova-funcionalidade
git push origin main
```

### 13.3. Limpeza do ambiente Docker

```
# Para containers mantendo dados
docker compose down

# Para e remove TUDO (containers + volumes + dados)
docker compose down -v

# Limpeza de imagens nao utilizadas
docker image prune -f
docker volume prune -f
```

## 14. Resolução de Problemas Comuns

---

### 14.1. Container db não inicia

```
docker compose logs db
# Erro comum: porta 5432 ja em uso
sudo ss -tlnp | grep 5432
# Solucao: mudar porta externa no docker-compose.yml
# ports: ["5434:5432"]
```

### 14.2. Celery não conecta ao Redis

```
# Verifica se o .env tem a variavel correta
cat .env | grep REDIS
# Deve conter: REDIS_URL=redis://redis:6379/0

# Testa conexao
docker compose exec backend python3 -c \
    "import redis; r=redis.from_url('redis://redis:6379/0'); \
    print('Redis OK:', r.ping())"

# Reinicia apos corrigir o .env
docker compose restart backend celery
```

### 14.3. PostGIS não disponível

```
docker compose exec db psql -U queimadas_user -d queimadas \
  -c "CREATE EXTENSION IF NOT EXISTS postgis;"
```

#### 14.4. Frontend não conecta ao backend

```
# Verifica CORS no settings.py
# CORS_ALLOWED_ORIGINS = ["http://localhost:5173"]

# Verifica variavel no frontend
cat frontend/.env
# VITE_API_BASE_URL=http://localhost:8000/api

# Testa o backend diretamente
curl http://localhost:8000/api/estatisticas/
```

#### 14.5. Importação falha silenciosamente

```
# Verifica os campos dispon veis no CSV
head -1 data/focos_2024.csv

# Verifica o que parse_linha espera
docker compose exec backend \
  grep -A 30 "def parse_linha" /app/queimadas/tasks.py
```

### 15. Checklist Final — Sistema Rodando

---

```
echo "=== CHECKLIST IGNISGEO ==="

echo -n "1. Containers rodando: "
docker compose ps --status=running \
  | grep -c "running" \
  && echo "OK (esperado: 5)" || echo "FALHOU"

echo -n "2. API respondendo: "
curl -s http://localhost:8000/api/estatisticas/ \
  | grep -q "total_focos" && echo "OK" || echo "FALHOU"

echo -n "3. Frontend acessivel: "
curl -s http://localhost:5173 \
  | grep -q "html" && echo "OK" || echo "FALHOU"

echo -n "4. Focos importados: "
curl -s http://localhost:8000/api/estatisticas/ \
  | python3 -c "import sys,json; \
    d=json.load(sys.stdin); \
    print(f'{d[\"total_focos\"]: ,} focos')"
```

```
echo -n "5. PostGIS ativo: "
docker compose exec db psql \
  -U queimadas_user -d queimadas -tAc \
  "SELECT postgis_version();" 2>/dev/null \
  | head -1 && echo "OK" || echo "FALHOU"

echo "=== FIM DO CHECKLIST ==="
```

#### Sistema operacional quando todos os itens retornam OK:

1. Containers rodando: 5 OK
2. API respondendo: OK
3. Frontend acessível: OK
4. Focos importados: 11.807.488 focos
5. PostGIS ativo: 3.4 USE\_GEOS=1 ... OK

## 16. Recuperação de Senha do Admin

```
# Lista usuarios cadastrados
docker compose exec backend python manage.py shell -c \
  "from django.contrib.auth.models import User; \
  print([u.username for u in User.objects.all()])"

# Troca a senha
docker compose exec backend \
  python manage.py changepassword admin
```

Acesse o painel administrativo em: <http://localhost:8000/admin/>

## 17. Resumo dos Endereços Locais

Table 1: Endereços e portas do IgnisGeo em desenvolvimento local.

| Serviço    | Endereço                                                                | Descrição                    |
|------------|-------------------------------------------------------------------------|------------------------------|
| Frontend   | <a href="http://localhost:5173">http://localhost:5173</a>               | Interface web Vue 3          |
| API Django | <a href="http://localhost:8000/api/">http://localhost:8000/api/</a>     | Todos os endpoints REST      |
| Admin      | <a href="http://localhost:8000/admin/">http://localhost:8000/admin/</a> | Painel administrativo Django |
| PostgreSQL | <a href="localhost:5433">localhost:5433</a>                             | Acesso externo ao banco      |
| Redis      | <a href="localhost:6379">localhost:6379</a>                             | Broker Celery                |

## References

---

- [1] INSTITUTO NACIONAL DE PESQUISAS ESPACIAIS (INPE). **BDQueimadas — Banco de Dados de Queimadas**. Disponível em: <https://queimadas.dgi.inpe.br>. Acesso em: abr. 2026.
- [2] DOCKER INC. **Docker Documentation**. Disponível em: <https://docs.docker.com>. Acesso em: abr. 2026.
- [3] POSTGIS DEVELOPMENT TEAM. **PostGIS 3.4 Documentation**. Disponível em: <https://postgis.net/docs>. Acesso em: abr. 2026.
- [4] CELERY PROJECT. **Celery 5 Documentation**. Disponível em: <https://docs.celeryq.dev>. Acesso em: abr. 2026.
- [5] CERQUEIRA, R. **IgnisGeo — Repositório**. Disponível em: <https://github.com/rogeriocerqueira/ignisGeo>. Acesso em: mai. 2026.